

Ejercicio 4 – Android - Splash Screen

Splash Screen es la primera pantalla visible para el usuario cuando se inicia la aplicación. La pantalla de bienvenida es una de las pantallas más vitales de la aplicación, ya que es la primera experiencia del usuario con la aplicación.

Al abrir la aplicación hay un momento de “nada” ya que la aplicación aún no está en memoria a esto se le llama “cold start”, el administrador de ventanas intenta dibujar una UI de marcador de posición utilizando elementos del tema de la aplicación como `windowBackground`. Entonces, en lugar de mostrar el valor predeterminado `windowBackground` (generalmente blanco o negro).

Esta puede ser cambiada a un dibujo personalizado que muestre su pantalla de inicio. De esta manera, la pantalla de inicio solo se muestra cuando es necesario y no ralentiza a los usuarios.

Vamos a crear una “ventana de presentación” y posteriormente, a los 5 sg en este caso, que aparezca la ventana principal.

1. Primero crea un nuevo proyecto en Android Studio del tipo “Empty Views Activity”

Llámale “**Splash_Screen**”, el Lenguaje “Kotlin” y aunque tenemos diversas plataformas y APIs en las que utilizar nuestra aplicación, nosotros nos centraremos en aplicaciones para teléfonos y tablets, en cuyo caso tan sólo tendremos que seleccionar la API mínima (es decir, la versión mínima de Android) que soportará la aplicación. Android 9.0. como versión mínima (API 28):

Una vez configurado todo pulsamos el botón **Finish** y Android Studio creará por nosotros toda la estructura del proyecto y los elementos indispensables que debe contener. Si todo va bien aparecerá la pantalla principal de Android Studio con el nuevo proyecto creado:

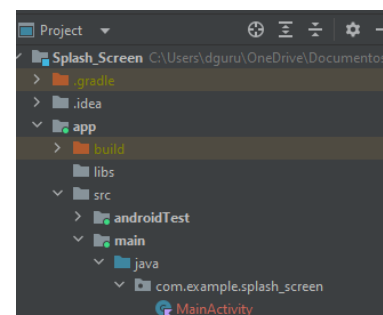
2. Recuerda las carpetas, archivos y contenidos principales de nuestro módulo principal.

1. Carpeta `/app/src/main/java`

Esta carpeta contendrá todo el código fuente de la aplicación, clases auxiliares, etc. Inicialmente, Android Studio creará por nosotros el código básico de la pantalla (actividad o activity) principal de la aplicación, que recordemos que en nuestro caso era **MainActivity**

2. Carpeta `/app/src/main/res/`

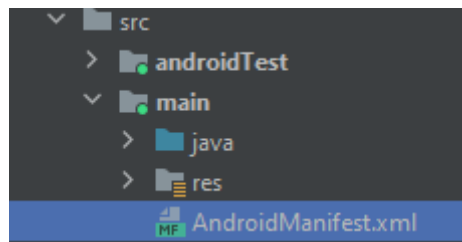
Contiene todos los ficheros de recursos necesarios para el proyecto: imágenes, layouts, cadenas de texto, etc. Los diferentes tipos de



recursos se pueden distribuir entre diversas subcarpetas. Entre los recursos creados por defecto cabe destacar los **layouts**, en nuestro caso sólo tendremos por ahora el llamado "**activity_main.xml**", que contienen la definición de la interfaz gráfica de la pantalla principal de la aplicación.

3. Fichero `/app/src/main/AndroidManifest.xml`

Contiene la definición en XML de muchos de los aspectos principales de la aplicación, como por ejemplo su identificación (nombre, icono, ...), sus componentes (pantallas, servicios, ...), o los permisos necesarios para su ejecución. Veremos más adelante más detalles de este fichero.



3. Creación de las Activities (ventanas)

Primero crea la actividad principal (MainActivity) que será la actividad a la que se dirigirá después de que la pantalla de bienvenida desaparezca.

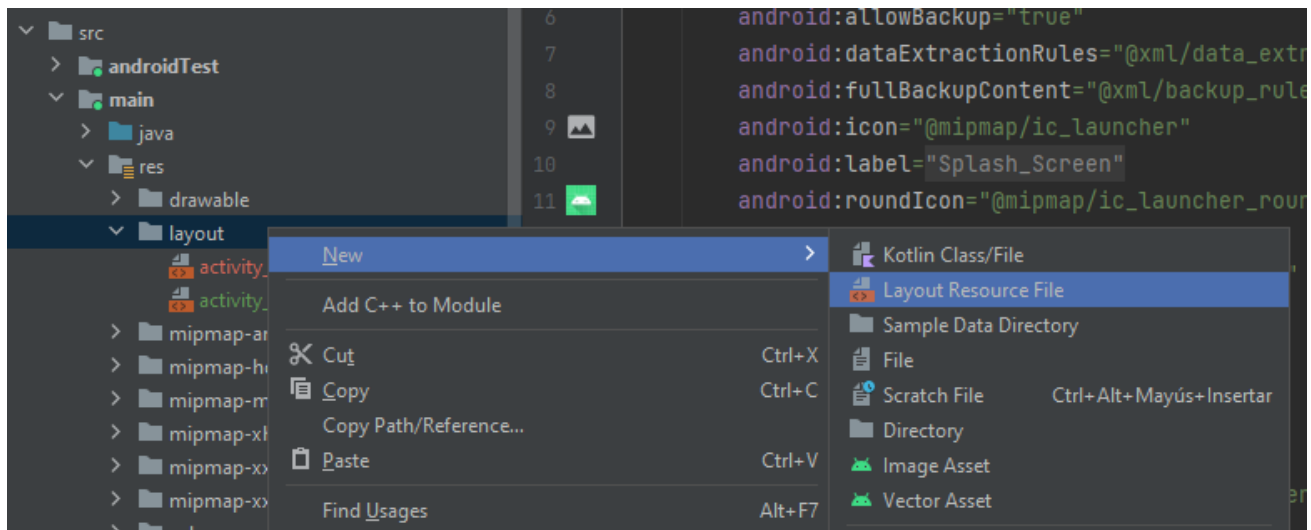
En la línea de "`android:text`", sustituye "`x.alumno.x`" por tu nombre. Luego configura el tamaño, negrita, y posición del texto:

```
activity_main.xml x MainActivity.kt x SplashActivit.kt x AndroidManifest.xml x activity_splash.xml x
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     xmlns:tools="http://schemas.android.com/tools"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     tools:context=".MainActivity">
8
9     <TextView
10         android:layout_width="wrap_content"
11         android:layout_height="wrap_content"
12         android:text="Actividad creada por x.alumno.x"
13         android:textSize="20dp"
14         android:textColor="#000000"
15         android:textStyle="bold"
16         app:layout_constraintBottom_toBottomOf="parent"
17         app:layout_constraintEnd_toEndOf="parent"
18         app:layout_constraintStart_toStartOf="parent"
19         app:layout_constraintTop_toTopOf="parent" />
20
21 </androidx.constraintlayout.widget.ConstraintLayout>
```

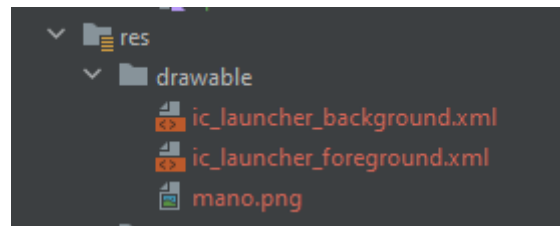
Debería quedarte así:



Ahora tienes que crear la nueva “activity” (ventana) en la carpeta “layout”. En la carpeta “layout” vete a “new” y selecciona “Layout Resource File”, y llámale “activity_splash”



Antes de pasar a configurar esta activity (será la ventana de bienvenida) descarga una imagen/dibujo de una mano saludando y déjalo en la carpeta “res/drawable” (tienes que ir a la carpeta del proyecto en Windows y luego recargar esa carpeta en Android Studio). Y ya deberías ver ese archivo:

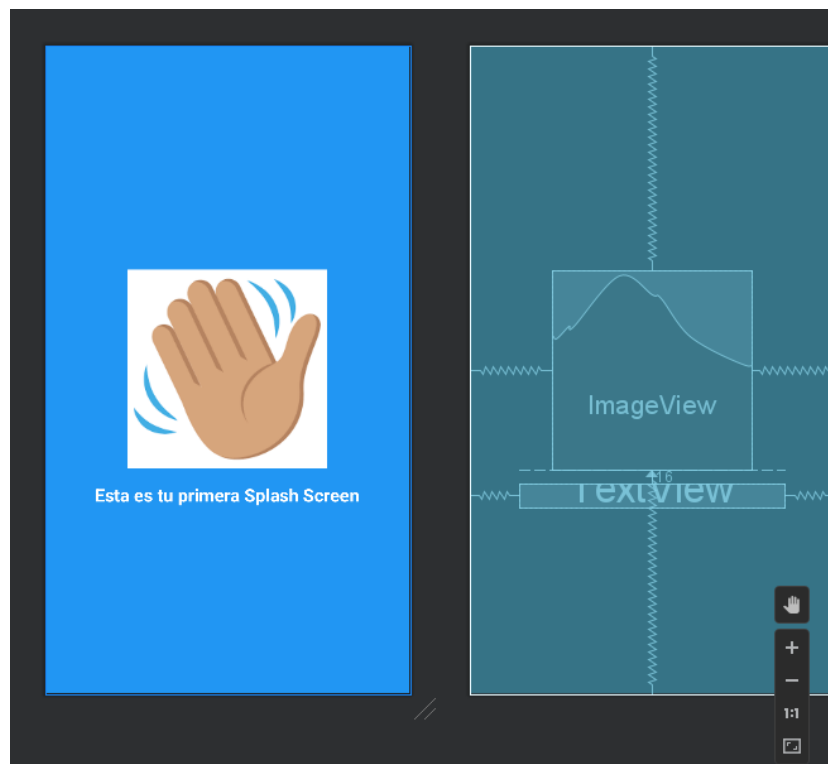


Ahora abre el archivo `activity_splash.xml` en la carpeta `res/layout` y define la interfaz de usuario. Incluye un **TextView** para el texto, un **ImageView** para la imagen de la mano y configura el fondo con el color azul. Todo integrado dentro de un mismo “**RelativeLayout**”

En el `TextView`, incluye el texto “Esta es tu primera Splash Screen”, negrita, centrado y tamaño **20sp** (se refiere a 20 puntos de escala (scale-independent pixels), son unidades de medida que escalan de acuerdo con la configuración de tamaño de fuente del dispositivo del usuario. Esto ayuda a garantizar que el tamaño del texto se vea adecuado en diferentes dispositivos con diferentes configuraciones de pantalla y tamaños de fuente).

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent">
5
6      <!-- activity_splash.xml -->
7      <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
8          android:layout_width="match_parent"
9          android:layout_height="match_parent"
10         android:background="#2196F3">
11
12         <ImageView
13             android:id="@+id/imageViewHand"
14             android:layout_width="wrap_content"
15             android:layout_height="wrap_content"
16             android:src="@drawable/mano"
17             android:layout_centerInParent="true"/>
18
19         <TextView
20             android:layout_width="wrap_content"
21             android:layout_height="wrap_content"
22             android:text="Esta es tu primera Splash Screen"
23             android:textSize="20sp"
24             android:textColor="#FFFFFF"
25             android:textStyle="bold"
26             android:layout_below="@id/imageViewHand"
27             android:layout_marginTop="16dp"
28             android:layout_centerHorizontal="true" /><!-- Ajusta esta línea -->
29
30     </RelativeLayout>
31
32 </androidx.constraintlayout.widget.ConstraintLayout>
```

Debería quedar algo así:



4. Ahora crearemos la lógica para que funcione la App

Ahora, crearemos la lógica para que la pantalla de bienvenida aparezca durante 5 segundos antes de pasar a la pantalla principal.

- Primero abre el archivo MainActivity.kt, para revisarlo:

```
activity_main.xml x MainActivity.kt x SplashActivit.kt x AndroidManifest.xml x
1 package com.example.splash_screen
2
3 import androidx.appcompat.app.AppCompatActivity
4 import android.os.Bundle
5
6 class MainActivity : AppCompatActivity() {
7     override fun onCreate(savedInstanceState: Bundle?) {
8         super.onCreate(savedInstanceState)
9         setContentView(R.layout.activity_main)
10    }
11 }
```

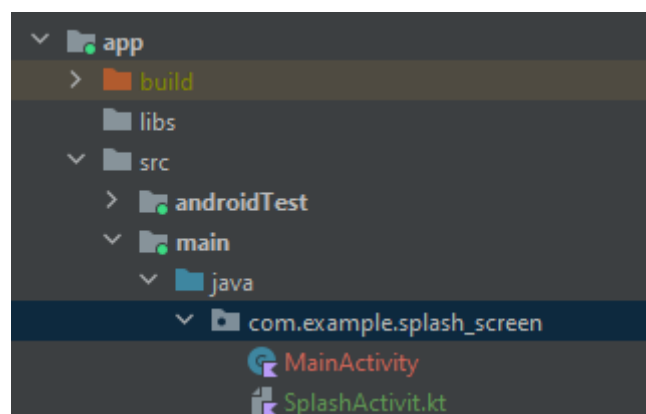
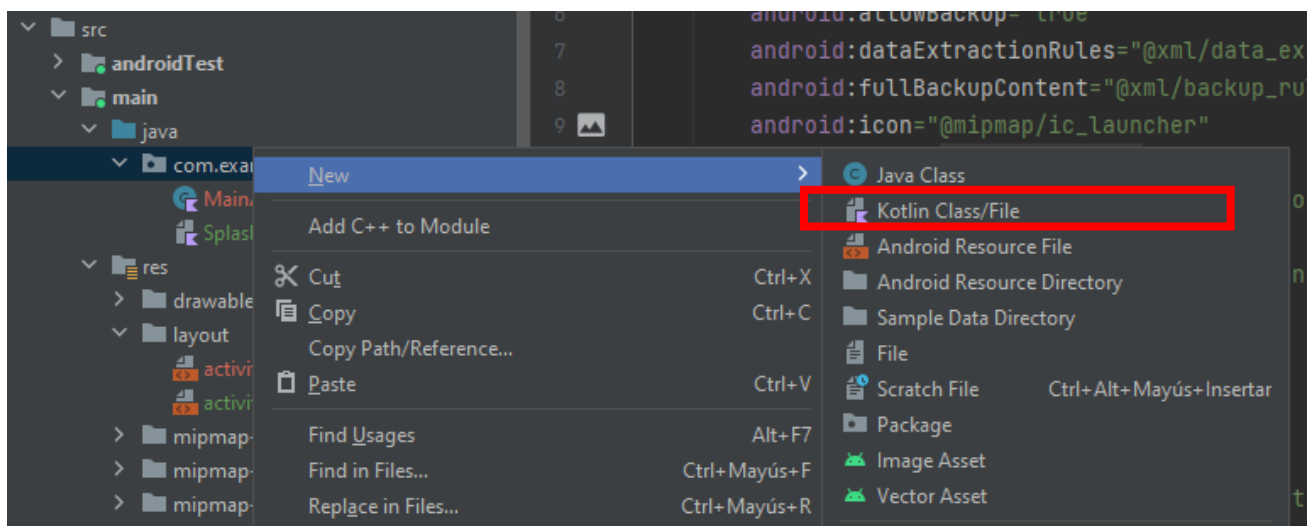
El método “onCreate” implementa la lógica de inicio de una actividad (su ciclo de vida) y se llama cuando la actividad se crea por primera vez. Con “override” se está reemplazando o sobrescribiendo el método “oncreate” y guardando los datos del estado anterior de la actividad.

Con “setContentView(R.layout.activity_main)” se establece la interfaz de usuario de la actividad actual, utilizando el diseño definido en el archivo XML activity_main.xml (ubicado en la ruta: /res/layout). Este archivo XML contiene la estructura y los elementos visuales (botones, textos, imágenes, etc.) que formarán la interfaz gráfica de la aplicación cuando se ejecute.

Este archivo no lo vamos a modificar de momento.

- Ahora, tienes que crear un nuevo archivo Kotlin para establecer la lógica de la activity_splash.

Para ello vete a las carpeta main/java/caom.example.splash_screen y crea el nuevo archivo, llámale “SplashActivity”:



Ahora, crea la lógica para que la pantalla de bienvenida aparezca durante 5 segundos (5000 milisegundo) antes de pasar a la pantalla principal. Abre el archivo SplashActivity.kt y agrega el siguiente código:

```
activity_main.xml x MainActivity.kt x SplashActivit.kt x AndroidManifest.xml x activity_splash.xml x
1 package com.example.splash_screen
2
3 import android.content.Intent
4 import android.os.Bundle
5 import android.os.Handler
6 import android.os.Looper
7 import androidx.appcompat.app.AppCompatActivity
8
9 new *
10 class SplashActivity : AppCompatActivity() {
11
12 new *
13 override fun onCreate(savedInstanceState: Bundle?) {
14     super.onCreate(savedInstanceState)
15     setContentView(R.layout.activity_splash)
16
17     // Usar el constructor de Handler que acepta un Looper
18     Handler(Looper.getMainLooper()).postDelayed({
19         // Crea un Intent para iniciar la actividad principal
20         val intent = Intent(packageContext, this@SplashActivity, MainActivity::class.java)
21         startActivity(intent)
22         finish()
23     }, delayMillis: 5000) // 5000 milisegundos = 5 segundos
24 }
25 }
```

Para ello, se ha utilizado **Handler**, que es una clase en Android que se utiliza para programar tareas (o mensajes) para ser ejecutadas en un hilo (thread) específico. En este caso lo hemos utilizado para crear un **Looper** (estructura de control que repite un conjunto de instrucciones hasta que se cumple una condición específica (que pasen los 5 segundos).

Y para ello hemos creado también un **intent** (recuerda que un "Intent" es un objeto que proporciona una descripción de una operación a realizar, como iniciar una actividad (Activity), enviar o recibir datos entre componentes de una aplicación, iniciar servicios, etc)

- El próximo paso va a ser hacer que la actividad "SplashActivity" esté registrada correctamente en el archivo **AndroidManifest.xml** y que esta sea la actividad de inicio.

Para ello tienes que modificar ese archivo cambiando los datos de la "activity_main" que es la que está configurada por defecto para inicializarse por los datos de la "activity_splash".

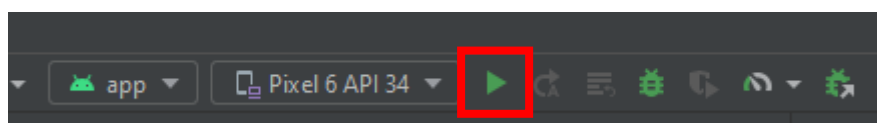
También tiene que estar registrada la "actividad" de la "MainActivity"

Tiene que quedar así:

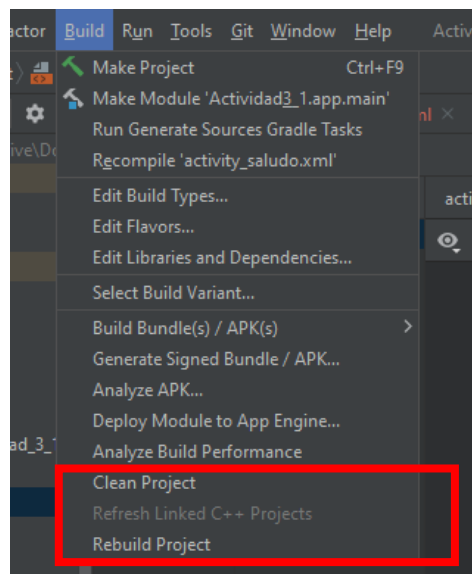
```
activity_main.xml x MainActivity.kt x SplashActivit.kt x MF AndroidManifest.xml x activity_splash.xml x
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools">
4
5     <application
6         android:allowBackup="true"
7         android:dataExtractionRules="@xml/data_extraction_rules"
8         android:fullBackupContent="@xml/backup_rules"
9         android:icon="@mipmap/ic_launcher"
10        android:label="@string/app_name"
11        android:roundIcon="@mipmap/ic_launcher_round"
12        android:supportRtl="true"
13        android:theme="@style/Theme.Splash_Screen"
14        tools:targetApi="31">
15        <activity
16            android:name=".SplashActivity"
17            android:exported="true">
18            <intent-filter>
19                <action android:name="android.intent.action.MAIN" />
20
21                <category android:name="android.intent.category.LAUNCHER" />
22            </intent-filter>
23        </activity>
24
25        <activity
26            android:name=".MainActivity"
27            android:label="@string/app_name">
28        </activity>
29
30    </application>
31
32 </manifest>
```

5. Compilar y probar el funcionamiento de la App en un dispositivo virtual

Para ver el aspecto y el comportamiento de tu app en un dispositivo, debes compilarla y ejecutarla. Android Studio configura proyectos para que puedas implementar tu app en un dispositivo virtual o físico. Si tuviéramos un dispositivo configurado podríamos ejecutar “Run App” para probar la App:



También es importante utilizar y conocer las opciones “Clean Project” y rebuild Project que nos ayudan a compilar el proyecto sin problemas (o ayudarnos a solucionarlos)



Clean y Rebuild Project

Clean Project (Limpiar Proyecto), limpia el proyecto eliminando todos los archivos compilados y los artefactos generados en la compilación anterior. Se utiliza cuando hay problemas en la compilación o cuando se quieren eliminar los archivos generados para comenzar desde cero. Puede ser útil después de realizar cambios significativos en el proyecto.

Rebuild Project (Reconstruir Proyecto), realiza la acción de "Clean Project" y luego realiza una compilación completa desde cero. Elimina todos los archivos compilados y regenera todo, incluyendo los archivos de clase, recursos y otros artefactos de construcción. Se utiliza cuando los problemas persisten incluso después de realizar la "Clean Project". Puede ser beneficioso cuando se sospecha que hay problemas profundos en la configuración del proyecto.

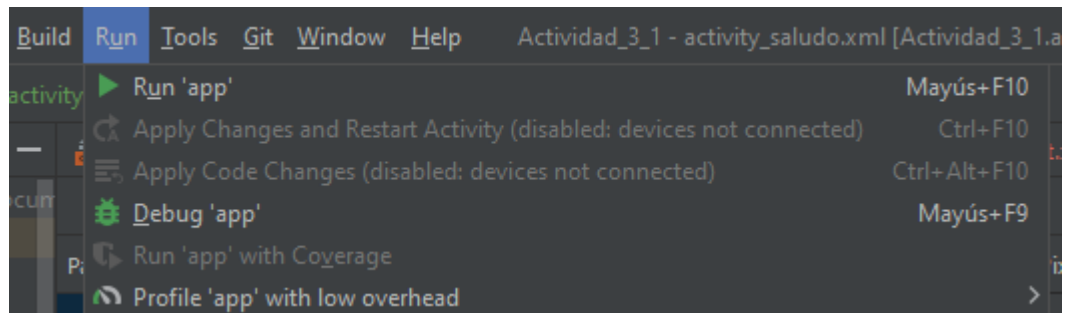
Si todos estos pasos se han ejecutado sin errores, guarda el proyecto y ahora vamos a **probar la aplicación Android** en nuestro PC sin tener que recurrir a un dispositivo físico, por lo que tenemos que definir lo que se denominan **AVD (Android Virtual Device)**. Para crear un AVD seleccionaremos el menú Tools / Android / AVD Manager.

el tamaño, resolución, y densidad de píxeles de su pantalla. En mi caso seleccionaré por ejemplo las características de un Pixel 6 y pasaremos al siguiente paso pulsando "Next".

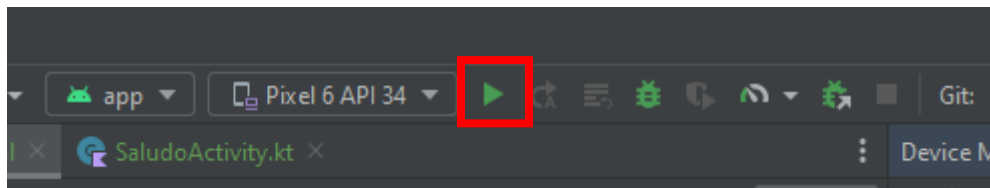
En el siguiente paso del asistente podremos configurar algunas características más del AVD, si simulará tener cámara frontal y/o trasera, teclado físico, ... Recomendando pulsar el botón "Show Advanced Settings" para ver todas las opciones disponibles. Si quieres puedes ajustar cualquiera de estos parámetros, pero por el momento os recomiendo dejar todas las opciones por defecto.

Recuerda que para ejecutar la App en el dispositivo que hemos creado podemos hacerlo de varias opciones:

1. Pulsaremos simplemente el menú **Run / ejecutar Run 'app'** (o la tecla rápida Mayús+F10). Android Studio.



2. Ejecutarlo desde la ventana de navegación de la parte superior izquierda:



Donde en caso de tener más dispositivos, podemos seleccionar el que queramos en ese momento. Le damos al botón de **“Run App”**

Si todo va bien, tras una pequeña (o no tan pequeña) espera aparecerá el emulador de Android y se iniciará automáticamente nuestra aplicación (si se inicia el emulador pero no se ejecuta automáticamente la aplicación podemos volver a ejecutarla desde Android Studio, mediante el menú Run, sin cerrar el emulador ya abierto).

Prueba a escribir un nombre y pulsar el botón "Aceptar" para comprobar si el funcionamiento es el correcto.